

Details and context for Tickets that did not contain this information in the document I sent out on April 6 2026

Each ticket is written for you all to build **without ambiguity** and aligns to your actual system architecture (ledger-first, ingestion → normalization → APIs → cap table).

BE-404 — OCR Structured Extraction (Alexandre)

Title:

BE-404 — OCR Structured Extraction (Documents → Structured Data)

Owner: Alexandre

Objective

Build OCR + AI extraction pipeline to convert uploaded documents into structured, validated JSON records.

Scope

Supported Document Types

- Lease
- Deed
- Mortgage Statement
- Insurance Policy

Inputs

- File upload (PDF, JPG, PNG)

Output (Mongo Schema)

```
DocumentRecord {  
  id  
  type  
  raw_text  
  extracted_fields  
  confidence_score  
  validation_status  
  created_at  
}
```

Core Functionality

1. OCR Layer

- Extract raw text using:
 - AWS Textract OR Tesseract OR Google Vision

2. AI Extraction

Extract fields:

- property_address
- tenant_name
- owner_name
- rent_amount
- lease_start / lease_end
- lender_name
- loan_balance
- insurance_provider

3. Normalization

- Dates → ISO format
- Currency → numeric
- Strings → trimmed

4. Validation

- Missing required fields → ERROR
- Low confidence → REVIEW
- Conflicts → FLAG

API Endpoints

- POST /api/ocr/upload
- GET /api/ocr/:id
- PATCH /api/ocr/:id/confirm

Acceptance Criteria

- Upload returns structured JSON
- Confidence score included
- Validation flags applied
- Admin can edit extracted fields

Edge Cases

- Multiple tenants
- Missing rent
- Low-quality scans

Dependencies

- None (foundational ingestion layer)

Definition of Done

- Document → structured JSON → stored in Mongo
- Validation system working
- API accessible by Property Registry

Title:

BE-112 — Property Registry API (System of Record)

Owner: Blessing

Objective

Create the **core system of record** for properties, units, and ownership.

Data Model

```
Property {  
  id  
  address  
  owner_entity_id  
  valuation  
  status  
}
```

```
Unit {  
  id  
  property_id  
  unit_number  
  tenant_id  
  rent  
}
```

Core Functionality**1. Property Management**

- Create property
- Update property
- Fetch property

2. Unit Management

- Add units to property

- Assign tenant to unit

3. Validation

- Unique address required
- Ownership must be defined

API Endpoints

- POST /api/properties
- GET /api/properties
- GET /api/properties/:id
- PATCH /api/properties/:id

Acceptance Criteria

- Property can be created
- Units can be attached
- Tenant linked to unit
- Full property tree returned

Dependencies

- OCR (optional input)
- Used by:
 - Ledger
 - Cap Table
 - Agreements

Definition of Done

- Property = root object across system
- All downstream systems reference property_id

BE-206 — Ownership Ledger (UPGRADE) (Abdul)

Title:

BE-206 — Ownership Ledger (Append-Only Financial System)

Owner: Abdul

Objective

Upgrade ledger to **append-only financial system** for token tracking.

Data Model

```
TokenLedgerEntry {  
  id  
  property_id  
  tenant_id  
  type  
  tokens  
  value  
  timestamp  
  source  
}
```

Entry Types

- accrual
- purchase
- adjustment
- forfeit

Core Logic

1. Append-Only

- No edits
- Only new entries allowed

2. Balance Calculation

balance = SUM(tokens)

3. TEPA Rules Enforcement

- No accrual if not in Good Standing
- Forfeit unvested tokens if violation
- Freeze tokens on eviction

(Aligned with TEPA contract logic)

API Endpoints

- POST /api/ledger/entry
- GET /api/ledger/:tenant_id
- GET /api/ledger/property/:id

Acceptance Criteria

- Ledger stores all events
- Balance derived (not stored)
- Cannot overwrite entries

Edge Cases

- Duplicate entries

- Negative token balance
- Retroactive adjustments

Dependencies

- Property Registry
- Stripe Integration

Definition of Done

- Ledger is source of truth for tokens
- Fully auditable history

BE-310 — Cap Table Engine (Abdullah)

Title:

BE-310 — Cap Table Engine (Ownership Calculation System)

Owner: Abdullah

Objective

Calculate ownership distribution across:

- landlord
- tenant
- investors

Data Model

```
CapTable {  
  property_id  
  total_tokens
```



```
allocations
}
```

Core Logic

Ownership %

$\text{ownership} = \text{tokens} / \text{total_tokens}$

Rules

- Cannot exceed 100%
- Must match ledger totals
- Tenant pool is fixed (anti-dilution)

API Endpoints

- GET /api/cap-table/:property_id
- POST /api/cap-table/recalculate

Acceptance Criteria

- Accurate % ownership
- Matches ledger balances
- Handles missing data

Edge Cases

- Over allocation (>100%)
- Missing valuation
- Token mismatch

Dependencies

- Ownership Ledger
- Property Registry

Definition of Done

- Cap table reflects real ownership state
- Fully consistent with ledger

BE-211 — Stripe Integration (Abdul)

Title:

BE-211 — Stripe Integration (Payments → Ledger)

Owner: Abdul

Objective

Integrate Stripe for:

- rent payments
- token purchases

Data Model

```
Payment {  
  id  
  tenant_id  
  amount  
  type  
  status  
  stripe_id  
}
```

Core Flows

1. Rent Payment

- Stripe → webhook → mark paid
- Trigger token accrual

2. Token Contribution

- Payment → convert to tokens
- Add ledger entry

Webhooks

- payment_intent.succeeded
- payment_failed

Endpoint:

- POST /api/stripe/webhook

Required Logic

- Idempotency (prevent duplicates)
- Retry handling
- Error logging

Acceptance Criteria

- Payment triggers ledger entry
- Failures handled
- No duplicate entries

Dependencies

- Ownership Ledger

Definition of Done

- Payments correctly reflected in ledger
- System stable under retries

BE-311 — Event Bus (Abdullah)

Title:

BE-311 — Event Bus (System-Wide Event Processing Layer)

Owner: Abdullah

Objective

Build a centralized **event-driven system** to handle all backend actions across KeyPath.

This ensures:

- systems stay decoupled
- actions trigger automatically
- no direct service-to-service dependency

Why This Matters (Critical)

Fact

Your system has multiple dependent flows:

- Stripe → Ledger → Cap Table
- Tenant Status → Ledger → Accrual
- OCR → Property → Agreements

Without Event Bus

- Tight coupling

- Bugs cascade across systems
- Hard to scale

With Event Bus

- Clean architecture
- Scalable
- Observable

Core Concept

Instead of:

Stripe → directly updates Ledger

You use:

Stripe → emits event → Event Bus → Ledger listens → Cap Table updates

Event Flow

Producer → Event Bus → Consumers

Tech Stack (MVP)

- Queue: Upstash Redis OR BullMQ
- Language: Node.js
- Pattern: Pub/Sub

(Aligned with backend stack)

Event Schema

```
Event {  
  id: string  
  type: string  
  payload: object
```

```
timestamp: number
source: string
status: "pending" | "processed" | "failed"
}
```

Core Events (MUST IMPLEMENT)

1. Payment Events

- payment.succeeded
- payment.failed

Payload:

```
{
  "tenant_id": "",
  "amount": 1000,
  "type": "rent" | "token"
}
```

2. Ledger Events

- ledger.entry.created

Payload:

```
{
  "tenant_id": "",
  "tokens": 1,
  "type": "accrual"
}
```

3. Cap Table Events

- cap_table.recalculate

4. Tenant Status Events

- tenant.good_standing.changed

5. OCR Events

- document.processed

Producers (Emit Events)

- Stripe Integration (BE-211)
- OCR Service (BE-404)
- Admin Actions
- Agreement Engine

Consumers (Listen to Events)

Event	Consumer
payment.succeeded	Ledger
ledger.entry.created	Cap Table
tenant.status.changed	Ledger
document.processed	Property Registry

Core Functionality

1. Emit Event

emitEvent(type, payload)

2. Subscribe to Event

subscribe("payment.succeeded", handler)

3. Process Queue

- Pull events from queue
- Execute handler
- Mark status

4. Retry Logic

- Retry failed events (max 3x)
- Move to “failed queue” if persistent

API (Internal)

- POST /internal/events/emit
- GET /internal/events/logs

Data Storage

Event Log Collection:

```
EventLog {  
  id  
  type  
  payload  
  status  
  retries  
  error  
  created_at  
}
```

Required Logic

Idempotency (CRITICAL)

- Prevent duplicate processing

- Use event_id tracking

Ordering

- Events for same entity must process sequentially

Failure Handling

- If event fails:
 - retry
 - log error
 - alert system

Acceptance Criteria

- Events can be emitted from any service
- Consumers process events correctly
- Retry logic works
- Failed events logged
- No duplicate execution

Edge Cases

- Duplicate Stripe webhook
- Event processed twice
- Out-of-order events
- Consumer crash mid-processing

Dependencies

- Used by:

- Stripe (BE-211)
- Ledger (BE-206)
- Cap Table (BE-310)
- OCR (BE-404)

Definition of Done

- Event system operational
- All major flows use events (not direct calls)
- Event logs visible for debugging
- System resilient to failure

● IMPORTANT IMPLEMENTATION NOTE

All future backend features MUST use event-driven flow.

No direct chaining like:

Stripe → Ledger → Cap Table

Only:

Stripe → Event Bus → Ledger → Event Bus → Cap Table

● FINAL ALIGNMENT NOTE

System Flow (Non-Negotiable):

OCR → Property Registry → Agreement → Ledger → Cap Table → Dashboard

- Ledger is the source of truth
- Cap table must match ledger
- Property is the root entity